

Diseño de un sistema de ensembles dinámicos multiobjetivo para entornos con concept drift

Trabajo Fin de Grado

Autor/a: **Eduardo Blázquez Verdejo**

Tutor/a:

Curso académico: **2025/2026**

Fecha: **Mayo de 2026**

Resumen

Tradicionalmente el aprendizaje automático parte de una hipótesis estacionaria: se entrena un modelo con un conjunto de datos y se espera que el modelo mantenga su rendimiento cuando se aplique sobre nuevas instancias. Si embargo, en la realidad, cuando los datos llegan de forma continua, la distribución subyacente puede transformarse con el tiempo. A este fenómeno se le conoce como *concept drift* y provoca que modelos entrenados sobre datos pasados, al enfrentarse a datos recientes, puedan ver su rendimiento degradado, incluso quedar obsoletos.

El presente Trabajo Fin de Grado plantea el diseño y evaluación de un sistema para la clasificación de flujos de datos con *concept drift*. En una primera fase se comparan distintos algoritmos de ensemble pasivos, entre ellos SEA, WAE, AUE2 y Learn++.NSE, con el fin de analizar su comportamiento en distintos escenarios de *drift*. Esta comparativa no se limita únicamente a la precisión de los modelos sino que incorpora distintas métricas relacionadas con la estabilidad, la recuperación tras el cambio y el coste computacional. A partir de esta evaluación se selecciona un algoritmo base sobre el que integrar un módulo de optimización multiobjetivo.

La propuesta principal consiste en utilizar un algoritmo evolutivo multiobjetivo, concretamente NSGA-II, como mecanismo de ajuste de la configuración del ensemble. El objetivo es explorar el compromiso entre precisión predictiva, robustez ante el *drift* y coste computacional. De esta forma el MOEA no se plantea como un clasificador independiente, si no como una capa de optimización encargada de seleccionar configuraciones adecuadas del sistema en función del rendimiento reciente.

Extended Abstract

Machine learning models are usually developed under the assumption that the data used during training and the data observed during deployment have a sufficiently similar distribution. This assumption allows the problem to be treated as a fixed supervised task: the model is trained on a finite dataset, validated on held-out observations, and then used to predict future instances. However many real world applications do not meet this assumption. Data may arrive continuously, the generating process may evolve, and the relationship between variables may change over time. In this type of context the model is exposed to a non-stationary environment. The phenomenon by which the data distribution or the target concept changes over time is generally referred as concept drift.

Concept drift is an important problem in stream learning because it directly affects the reliability of previously learned knowledge. A classifier that performed well on past observations can become obsolete in new examples generated by a different concept. This degradation can happen in many different ways, sudden, progressive, recurrent or even mixed, depending on how the data distribution changes.

Índice

1	Introducción	1
1.1	Contexto	1
1.2	Motivación	1
1.3	Estructura del trabajo	2
2	Objetivos	3
3	Estado del arte y preliminares	4
3.1	Aprendizaje en flujos de datos	4
3.2	Concept drift	5
3.3	Tipos de concept drift	6
3.4	Estrategias de adaptación al concept drift	7
3.5	Ensembles para streams no estacionarios	8
3.6	Algoritmos de ensembles pasivos chunk-based	9
3.6.1	SEA	9
3.6.2	AUE2	9
3.6.3	WAE	10
3.6.4	Learn++.NSE	10
3.7	Optimización multiobjetivo	11
3.8	NSGA-II	12
4	Metodología	14
4.1	Planteamiento general	14
4.2	Escenarios de evaluación	15
4.3	Evaluación prequential y Modelos desarrollados	17
4.4	Métricas de evaluación	19
4.5	Criterio de selección del algoritmo base	21
4.6	Módulo de optimización multiobjetivo	22
5	Implementación y evaluación experimental	23
5.1	Diseño e implementación de los ensembles pasivos	23
5.2	Comparación de los ensembles pasivos	28
5.3	Selección del algoritmo base	31
5.4	Implementación del MOEA	31
5.5	Resultados del sistema optimizado	35
5.6	Discusión de los resultados	37
6	Conclusiones	37
7	Vías futuras	37

1. Introducción

1.1. Contexto

En los últimos años, el aprendizaje automático se ha aplicado a un gran número de problemas en los que los datos no se encuentran disponibles de forma estática, sino que llegan de manera continua a lo largo del tiempo. Este tipo de escenarios aparece, por ejemplo, en sistemas de monitorización, redes de sensores, transacciones financieras, registros de actividad de usuarios o aplicaciones en las que las decisiones deben actualizarse conforme se reciben nuevas observaciones. A diferencia del aprendizaje supervisado tradicional, en el que normalmente se parte de un conjunto de entrenamiento fijo y disponible de antemano, el aprendizaje en flujos de datos obliga a procesar la información de forma incremental. En estos entornos, los modelos deben ser capaces de realizar predicciones con la información disponible hasta el momento y actualizarse sin necesidad de almacenar todo el histórico de datos. Estas restricciones introducen nuevos retos relacionados con el tiempo de procesamiento, el uso de memoria y la capacidad de adaptación. Uno de los principales problemas asociados a este contexto es la deriva de concepto o *concept drift*. Este fenómeno se produce cuando la distribución que genera los datos cambia con el tiempo, de modo que el conocimiento aprendido a partir de observaciones pasadas puede dejar de ser válido para clasificar las instancias futuras. Por lo que un modelo que inicialmente obtiene buenos resultados puede sufrir una degradación progresiva o repentina de su rendimiento.

Para abordar este problema, una familia de métodos especialmente relevante son los *ensembles* o conjuntos de clasificadores. Estos métodos combinan varios modelos base, normalmente entrenados en distintos momentos del flujo, y permiten incorporar nuevo conocimiento, reducir la influencia de clasificadores obsoletos y mejorar la robustez del sistema frente a cambios en la distribución de los datos. Sin embargo, el rendimiento de estos sistemas depende de múltiples decisiones de diseño, como la forma de actualizar los clasificadores, la estrategia de ponderación, el tamaño del conjunto o los hiperparámetros del modelo base. En este contexto, la optimización multiobjetivo ofrece un marco adecuado para estudiar configuraciones que no solo maximicen el rendimiento predictivo, sino que también tengan en cuenta otros criterios relevantes, como la estabilidad, la diversidad del conjunto o el coste computacional.

Este Trabajo Fin de Grado se sitúa en esta línea, proponiendo el diseño y evaluación de un sistema basado en ensembles pasivos y optimización multiobjetivo para entornos con *concept drift*.

1.2. Motivación

La motivación principal de este trabajo surge de la dificultad de mantener modelos predictivos fiables en entornos no estacionarios. En muchos problemas reales, no basta con entrenar un modelo una única vez y asumir que seguirá funcionando correctamente en el futuro. Si el

proceso que genera los datos cambia, el modelo debe adaptarse para no quedar inútil. Los algoritmos de *ensemble* aplicados a flujos de datos proporcionan una solución interesante a este problema, ya que permiten combinar clasificadores entrenados sobre diferentes bloques del flujo y ajustar su influencia conforme cambia el entorno. No obstante, cada algoritmo presenta un comportamiento distinto ante diferentes tipos de deriva. Algunos métodos pueden reaccionar mejor ante cambios abruptos, mientras que otros pueden conservar mejor el conocimiento en escenarios recurrentes o ser más estables durante transiciones graduales. Por este motivo, resulta necesario comparar varios algoritmos bajo las mismas condiciones experimentales y utilizando métricas que vayan más allá de la precisión media. Además, el rendimiento de un *ensemble* pasivo depende en gran medida de su configuración. Parámetros como la forma de ponderar los clasificadores, el tamaño del conjunto o los hiperparámetros de los clasificadores base pueden afectar de manera significativa al equilibrio entre precisión y coste. Esta observación motiva la incorporación de un módulo de optimización multiobjetivo, cuyo objetivo es ajustar dinámicamente la configuración del algoritmo seleccionado en función del rendimiento observado en los bloques recientes.

La tarea principal del trabajo consiste, por tanto, en estudiar si un algoritmo evolutivo multiobjetivo puede actuar como una capa de ajuste sobre un *ensemble* pasivo, permitiendo mejorar el compromiso entre rendimiento predictivo, diversidad y coste computacional sin sustituir al clasificador base.

1.3. Estructura del trabajo

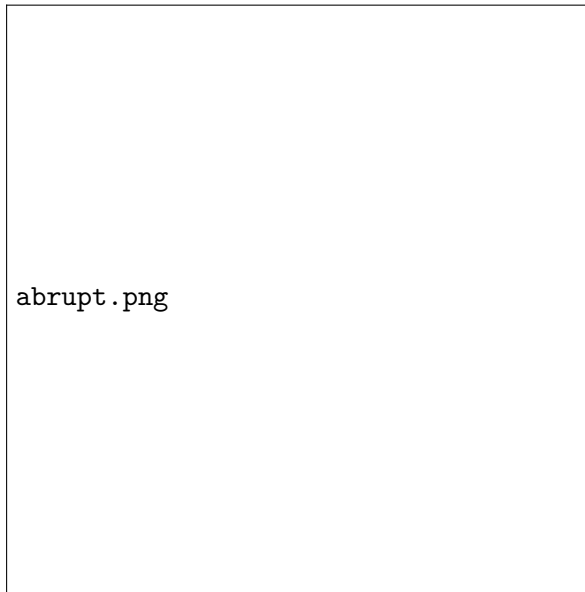
El resto del documento se organiza de la siguiente forma.

En la Sección 2 se presentan el objetivo general y los objetivos específicos del Trabajo Fin de Grado.

En la Sección 3 se desarrolla el estado del arte y los conceptos preliminares necesarios para contextualizar el trabajo. En primer lugar, se introduce el aprendizaje en flujos de datos y el problema del *concept drift*. A continuación, se describen distintos tipos de deriva, estrategias de adaptación y algoritmos de *ensemble* para entornos no estacionarios. Finalmente, se presentan los fundamentos de la optimización multiobjetivo y del algoritmo NSGA-II.

En la Sección 4 se expone la metodología seguida, se describen los escenarios de evaluación generados, el esquema de evaluación prequential, los modelos comparados, las métricas utilizadas y el criterio seguido para seleccionar el algoritmo base. También se presenta el diseño metodológico del módulo de optimización multiobjetivo.

En la Sección 5 se detalla el diseño e implementación del sistema propuesto. Se explican la arquitectura general, la implementación de los algoritmos base, el cálculo de las métricas, la ejecución de los experimentos y la integración del módulo MOEA con el algoritmo Learn++.NSE. En la Sección 5 también se presentan y analizan los resultados experimentales obtenidos. En primer lugar, se comparan los algoritmos pasivos en los distintos escenarios de



deriva. Posteriormente, se estudia el comportamiento del sistema optimizado y se discuten los resultados en términos de rendimiento, recuperación, diversidad y coste.

Por último, en la Sección 6 se recogen las conclusiones principales del trabajo y en la Sección 7 se plantean posibles líneas de mejora y ampliación futura.

2. Objetivos

El objetivo general de este Trabajo Fin de Grado es diseñar y evaluar un sistema de *ensembles* para clasificación en flujos de datos con *concept drift*, incorporando un módulo de optimización multiobjetivo que permita ajustar dinámicamente la configuración del algoritmo base seleccionado. A partir de este objetivo general, se plantean los siguientes objetivos específicos:

- Estudiar el problema del aprendizaje en flujos de datos no estacionarios y analizar el impacto del *concept drift* sobre el rendimiento de los modelos de clasificación.
- Revisar distintos algoritmos de pasivos aplicados a flujos de datos: SEA, AUE2, WAE y Learn++.NSE.
- Diseñar escenarios sintéticos de evaluación que permitan simular distintos tipos de deriva de concepto, concretamente deriva abrupta, gradual y recurrente.
- Evaluar los algoritmos considerados utilizando métricas relacionadas con el rendimiento predictivo, la estabilidad, la recuperación tras el cambio, la diversidad del *ensemble* y el coste computacional.
- Seleccionar un algoritmo base a partir de los resultados obtenidos para ser optimizado mediante un enfoque multiobjetivo.

- Diseñar e integrar un MOEA para ajustar los hiperparámetros del algoritmo seleccionado durante el procesamiento del flujo.
- Analizar si la optimización multiobjetivo permite mejorar el compromiso entre precisión, diversidad y coste computacional frente al uso de una configuración fija.

3. Estado del arte y preliminares

3.1. Aprendizaje en flujos de datos

El aprendizaje en flujos de datos (data stream mining) es una respuesta a la necesidad de procesar información en escenarios en los que los datos no se encuentran disponibles de forma finita, si no que llegan de forma continua a lo largo del tiempo. Podemos definir formalmente flujo de datos como una secuencia temporal ordenada de instancias, potencialmente infinita, en la que el sistema solo dispone de la información observada hasta un momento dado y ha de ser capaz de aplicar el conocimiento aprendido a instancias futuras inmediatamente.[1]

Este ámbito de estudio se diferencia en gran medida del aprendizaje supervisado clásico por lotes o batch learning, en que en el enfoque tradicional asumimos que el conjunto de entrenamiento es finito, estático y está disponible en su totalidad de antemano, lo que permite que los algoritmos lo procesen durante múltiples pasadas durante la fase de entrenamiento. En flujos de datos la instancias que llegan de forma secuencial no se pueden almacenar indefinidamente, ni ser reanalizadas varias veces. Además, mientras que el aprendizaje tradicional suele apoyarse en la hipótesis de que los datos de entrenamiento y prueba proceden de una distribución fija e independiente (i.i.d), los flujos de datos pertenecen a entornos dinámicos donde este principio de estacionariedad deja de cumplirse. [2]

Esta continuidad y volumen de datos en los flujos hacen que aparezcan grandes restricciones computacionales y arquitectónicas. En primer lugar los algoritmos están sujetos a límites estrictos de memoria, ya que conservar todo el historial de datos resulta tanto física como económicamente inviable. En segundo lugar cada instancia tiene que procesarse en un tiempo reducido, ya que el sistema no controla el orden de llegada de los datos, que pueden llegar en cualquier momento. Por último los métodos utilizados deben poseer la capacidad de aprender de forma incremental, manteniendo el modelo actualizado. [1], [3]

Existen distintos tipos de métodos para el procesamiento de un flujo de datos dependiendo de la gestión de tiempo y memoria. Como el procesamiento *online* o el basado en bloques, el uso de ventanas o los *ensembles*. Esto se desarrollará en los proximos apartados.

En conjunto, este aprendizaje exige modelos que cumplan con el dilema de la estabilidad-plasticidad, entendemos por estabilidad la retención de conocimiento útil y relevante y por plasticidad la capacidad de aprender nuevos conceptos [4]. Esta necesidad se vuelve especialmente relevante cuando el proceso que genera los datos cambia con el tiempo, dando lugar al

problema conocido como *concept drift* o deriva de concepto.

3.2. Concept drift

La deriva de concepto o *concept drift* surge por los cambios imprevistos y no estacionarios en la distribución subyacente de los flujos de datos a lo largo del tiempo [5]. En el aprendizaje tradicional los modelos operan en entornos estáticos, sin embargo en el mundo real, los procesos que generan datos están sujetos a evoluciones constantes.

Formalmente el *concept drift* aparece cuando las propiedades estadísticas de la variable objetivo o *target*, que el modelo intenta predecir varían de manera inesperada. Esto es la causa principal de la pérdida de efectividad en sistemas de predicción [5].

Desde una perspectiva bayesiana, podemos definir un *concepto* como la distribución de probabilidad conjunta de las características de entrada y la etiqueta de clase en un instante. Formalmente, existe *concept drift* entre dos instantes en el tiempo t y $t + 1$ si se cumple que:

$$P_t(\mathbf{X}, y) \neq P_{t+1}(\mathbf{X}, y)$$

donde $\mathbf{X}$ representa el vector de características de entrada e y la etiqueta de clase. Esta probabilidad conjunta puede descomponerse como:

$$P_t(\mathbf{X}, y) = P_t(\mathbf{X}) P_t(y | \mathbf{X})$$

En esta expresión, $P_t(\mathbf{X})$ representa la distribución marginal de las características de entrada, mientras que $P_t(y | \mathbf{X})$ representa la probabilidad posterior de la clase dado el vector de características. Un cambio en $P_t(\mathbf{X})$ puede alterar la distribución de las instancias observadas en el espacio de características, mientras que un cambio en $P_t(y | \mathbf{X})$ modifica directamente la relación entre las entradas y la clase objetivo, pudiendo invalidar los límites de decisión ya aprendidos [5], [6].

El *concept drift* rompe con la hipótesis de una distribución fija y estacionaria (i.i.d). Esto significa que el modelo es evaluado sobre una distribución que ya no coincide con la que se usó en un principio para construirlo.

La presencia de este fenómeno hace que sea necesario que los modelos sean capaces de actualizarse en tiempo real para mantener su rendimiento. Sin mecanismos de actualización, un sistema de aprendizaje en flujos de datos quedaría obsoleto tras el primer cambio en el entorno. Por tanto, los algoritmos deben estar equipados con mecanismos que les permitan olvidar información no útil y reajustar sus fronteras de decisión conforme el concepto evoluciona [7].

3.3. Tipos de concept drift

Podemos categorizar el *concept drift* atendiendo a: la naturaleza estadística del cambio o el patrón de transición temporal.

Clasificación según la naturaleza del cambio.

- **Deriva real (*real concept drift*):** se produce cuando cambia la probabilidad posterior de las clases

$$P_t(y | \mathbf{X}) \neq P_{t+1}(y | \mathbf{X})$$

Este tipo de deriva es el más crítico, ya que altera directamente los límites de decisión del modelo, provocando que el conocimiento sea erróneo [6].

- **Deriva virtual (*virtual drift*) o *covariate shift*:** aparece cuando cambia la distribución de las características de entrada, pero se mantiene estable la relación entre las entradas y la clase objetivo. Formalmente, puede expresarse como:

$$P_t(\mathbf{X}) \neq P_{t+1}(\mathbf{X})$$

manteniéndose, en principio:

$$P_t(y | \mathbf{X}) = P_{t+1}(y | \mathbf{X})$$

En este caso, los límites de decisión no tienen por qué cambiar necesariamente. Sin embargo, el modelo puede perder utilidad en las nuevas regiones pobladas del espacio de búsqueda [6].

Clasificación según el patrón de transición

- **Abrupto:** El concepto viejo es reemplazado instantáneamente por uno nuevo en un momento determinado. Podría ser por ejemplo reemplazar un sensor por otro que tiene una calibración distinta [2].

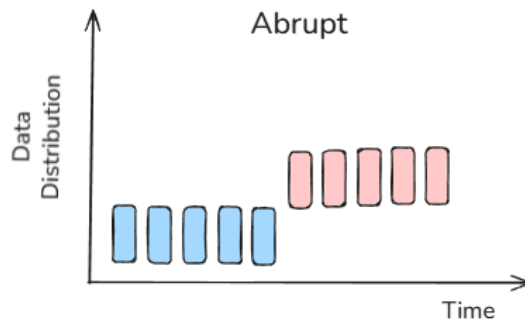


Figura 1: Drift Abrupto.

- **Gradual:** Se caracteriza por un periodo en el que el concepto antiguo y el nuevo se alternan. Durante este tiempo, la probabilidad de observar instancias del concepto anterior disminuye de forma progresiva mientras aumenta la del nuevo.[1]

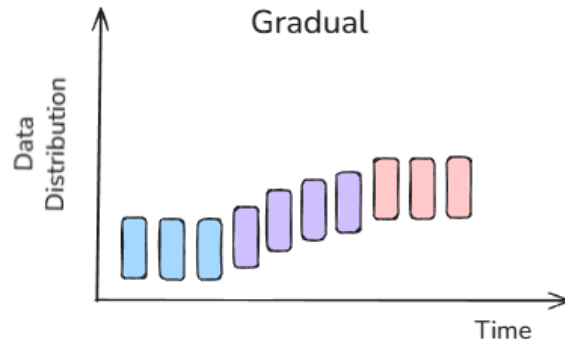


Figura 2: Drift Gradual.

- **Recurrente:** Se produce cuando un concepto que ya había aparecido anteriormente vuelve a ser relevante con el tiempo. Este tipo de deriva suele ser cíclica si los conceptos recurren en un orden específico [6].

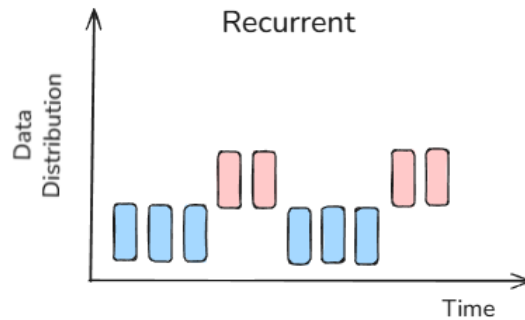


Figura 3: Drift Recurrente.

3.4. Estrategias de adaptación al concept drift

Las estrategias para hacer frente al *concept drift* tienen como objetivo fundamental preservar la validez y precisión predictiva de los modelos ante los cambios en la distribución subyacente de los datos. Estas técnicas buscan resolver el dilema de la estabilidad-plasticidad, equilibrando la retención de conocimiento útil con la capacidad de integrar rápidamente nuevos patrones detectados en el flujo.

Los dos enfoques principales de adaptación al *concept drift* son:

1. **Adaptación activa:** estas estrategias dependen de mecanismos de detección de drift explícitos que vigilan el rendimiento o cambios en la distribución de los datos. Cuando el

detector emite una señal de alarma, el sistema toma ciertas acciones como el reentrenamiento o una nueva estructuración del modelo para mitigar la caída de la precisión. Permiten tomar medidas rápidas frente cambios abruptos pero son susceptibles a falsas alarmas. Como por ejemplo ADWIN [8].

2. **Adaptación pasiva:** Estos métodos asumen que la deriva puede ocurrir de forma continua y no dependen de un detector. El modelo se actualiza de manera ininterrumpida con cada nuevo bloque de datos. La principal limitación es que el olvido puede ser lento, lo que resulta, en algunas situaciones, una adaptación lenta ante cambios bruscos o pérdida de estabilidad en periodos más estables.

Además de esta distinción, las estrategias de adaptación pueden apoyarse en mecanismos de gestión de memoria y olvido. Las ventanas por ejemplo permiten conservar los datos más recientes, los factores de desvanecimiento reducen progresivamente el peso de la información antigua y en otros casos la adaptación se hace mediante la sustitución completa del modelo o la actualización parcial de alguna de sus partes.

Otra forma de adaptación son los conjuntos de clasificadores o ensembles. Estos permiten combinar modelos entrenados en distintos momentos del flujo, ajustar sus pesos, incorporar nuevos clasificadores o reducir la influencia de los que se han quedado obsoletos. Es por eso que constituyen una buena alternativa para escenarios no estacionarios. [2], [9]

3.5. Ensembles para streams no estacionarios

Los *ensembles* o conjuntos de clasificadores son una de las herramientas más potentes y flexibles para los problemas de clasificación en flujos de datos no estacionarios. Los *ensembles* permiten gestionar el conocimiento de forma modular, facilitando que se incorpore nueva información y olvidando patrones no útiles de manera selectiva.

La principal motivación para el uso de *ensembles* es el teorema de "*no free lunch theorem*", que estipula que ningún algoritmo de aprendizaje es óptimo para todas las tareas posibles, cada cual es competente en un dominio. [1]

Según la forma en la que procesan los datos podemos distinguir dos tipos de *ensembles*:

- **Chunk-based:** los datos se procesan en bloques de tamaño fijo y la construcción, evaluación o actualización de los clasificadores no se realiza hasta que todas las muestras de un nuevo bloque están disponibles. Algunos ejemplos que se tratarán en los próximos apartados: *SEA*, *AUE*, *Lean++*, *NSE*.
- **Online:** estos modelos se actualizan instancia a instancia. Son ideales para aplicaciones en las que hay restricciones estrictas de memoria y tiempo, ya que procesan cada ejemplo en cuanto disponen de él y una única vez. Podemos destacar *Online Bagging* y *DMW*.

También los *ensembles* pueden pertenecer a cualquiera de los dos bloques de los se ha hablado en el apartado anterior, pueden ser tanto activos como pasivos.

La estructura en la que se organizan los clasificadores también define las capacidades del conjunto. La estructura más ampliamente usada es la plana o *flat*, en esta organización los modelos base se entrenan de forma independiente y luego se fusiona su conocimiento con una función de combinación simple como por ejemplo la votación ponderada.

Otros tipos de estructuras aunque menos usadas podrían ser: el meta-aprendizaje que utiliza un meta-clasificador entrenado con las salidas de los modelos de primer nivel o la estructura en redes que organiza sus clasificadores como nodos en un grafo, ponderando su influencia según métricas de centralidad o similitud entre predicciones [9].

3.6. Algoritmos de ensembles pasivos chunk-based

3.6.1. SEA

El *Streaming Ensemble Algorithm* (SEA) es uno de los trabajos fundacionales en el uso de *ensembles* [10]. SEA procesa el flujo en *chunks*. Para cada nuevo bloque, se entrena un clasificador candidato y se evalúa junto con el resto del *ensemble*. Si el conjunto no ha alcanzado aún el tamaño máximo, el clasificador se añade directamente. En caso de que haya alcanzado el límite se aplica una política de reemplazo: se sustituye un clasificador existente que tenga peor rendimiento.

La predicción se obtiene mediante votación mayoritaria no ponderada, por lo que todos los clasificadores tienen la misma influencia en la decisión final. A diferencia de otros algoritmos posteriores, SEA no ajusta pesos individuales, su adaptación a la deriva se hace exclusivamente mediante actualización: clasificadores obsoletos son sustituidos por nuevos modelos entrenados con datos recientes.

Principalmente destaca por su eficiencia computacional y el bajo consumo de memoria. Sin embargo el no votar de forma ponderada puede hacer que tarde en adaptarse al *drift*, y se depende sobretodo de la elección del tamaño de bloque: bloques grandes retrasarán la reacción al cambio y bloques demasiado pequeños generarán clasificadores débiles.

3.6.2. AUE2

El algoritmo *Accuracy Updates Ensemble* (AUE2) representó una evolución en el campo del *data stream mining* ya que propone una estructura híbrida que combina procesar por bloques junto a actualizaciones incrementales de sus modelos. Este algoritmo nace con la idea de superar las limitaciones de sus predecesores (SEA y AWE) que reaccionan lento a derivas abruptas y dependen en exceso del tamaño de bloque [3].

Los autores argumentan que aunque los métodos online son rápidos ante cambios bruscos, no aprovechan los mecanismos de pesado que benefician la adaptación a cambios lentos; al contrario, los métodos por chunks sufren demasiado si el cambio ocurre dentro de un mismo bloque. AUE2 resuelve esto combinando mecanismos de pesado con la naturaleza incremental de los *Árboles de Hoeffding*.

Por cada nuevo bloque, AUE2, entrena un clasificador candidato utilizando la información de dicho bloque y se evalúan todos los clasificadores sobre el *ensemble* actual. Tras esta evaluación se actualizan los pesos de los miembros del conjunto según su error de predicción, de esta manera los clasificadores con mejor rendimiento tienen mayor influencia en la decisión final. La predicción se obtiene mediante votación ponderada.

Una de las características principales de AUE2 es que actualiza los miembros del *ensemble* incrementalmente con las instancias del bloque actual. Esto es lo que hace que se reduzca la dependencia al tamaño del bloque ya que los clasificadores se ajustan de forma continua a las nuevas distribuciones.

3.6.3. WAE

Weighted Aging Ensemble (WAE), a diferencia de SEA, incorpora un mecanismo de envejecimiento de clasificadores y el uso de medidas de diversidad para la poda del conjunto [11].

A diferencia de de otros métodos que ponderan exclusivamente por precisión, WAE asigna pesos basándose también en la antigüedad en el conjunto. El objetivo es que el olvido sea suave: los modelos que han sobrevivido mucho tiempo pierden gradualmente su influencia. Cuando el conjunto alcanza el tamaño máximo evalúa los subconjuntos candidatos y selecciona el que maximice una medida de diversidad. Esta técnica busca mejorar la robustez global.

El rejuvenecimiento, técnica que aplica WAE, permite reducir artificialmente el contador de antigüedad de un clasificador si este tiene un impacto muy alto en la precisión del conjunto. Esta técnica está especialmente diseñada para escenarios recurrentes, permitiendo que modelos antiguos recuperen su peso.

3.6.4. Learn++.NSE

El algoritmo *Learn++.NSE* (*Nonstationary Environments*) es una de las arquitecturas más robustas y versátiles entre los *ensembles*. A diferencia de sus predecesores de la familia Learn++, este fue diseñado específicamente para enfrentar el dilema de la estabilidad-plasticidad [4].

Al igual que los algoritmos vistos anteriormente, Learn++.NSE entrena un nuevo clasificador por lote y los clasificadores se evalúan sobre el lote para determinar su relevancia actual. Lo que diferencia a Learn++.NSE es su mecanismo de pesado dinámico basado en el tiempo. El

peso de voto de cada clasificador depende de una media ponderada de su precisión a lo largo del tiempo, que se ajusta mediante una función sigmoideal.

Antes de entrenar un nuevo clasificador Learn++.NSE calcula una medida, el error del conjunto existente. El objetivo es generar una distribución de penalización que guiará como se evalúan los modelos individuales. La función sigmoideal sirve para dar más pesos a los errores cometidos recientemente y reducir el peso de errores antiguos. Si un modelo antiguo ya no es relevante su error ponderado aumenta y la función sigmoide lo "silenciará", pero también permite que se vuelva a activar si aparece de nuevo el concepto antiguo.

3.7. Optimización multiobjetivo

La optimización multiobjetivo permite modelar problemas complejos de optimización. En escenarios reales donde los objetivos considerados entran en conflicto entre sí, por lo que conseguir mejorar una métrica hace que en la mayoría de casos empeoren otras. Para un problema multiobjetivo la solución pasa por analizar un conjunto de soluciones, en la que cada una satisface los objetivos en un nivel aceptable sin estar dominada por una solución en concreto [12].

Formalmente, un problema de optimización multiobjetivo con K objetivos se define como la búsqueda de un vector de variables de decisión $\mathbf{x}^*$ dentro de un espacio de soluciones factibles X , de forma que se minimice o maximice un conjunto de funciones objetivo:

$$\mathbf{z}(\mathbf{x}) = (z_1(\mathbf{x}), z_2(\mathbf{x}), \dots, z_K(\mathbf{x}))$$

A diferencia de la optimización escalar, donde existe una única solución óptima, en estos problemas se busca un conjunto de soluciones que representen los mejores intercambios posibles entre los distintos objetivos.

Dado que es extremadamente complicado encontrar la solución perfecta que optimice todos los objetivos a la vez, se busca algo llamado soluciones de compromiso. El proceso de selección de estas soluciones suele realizarse mediante técnicas de toma de decisiones multicriterio una vez obtenido un conjunto de soluciones no dominadas. Métodos como la programación de compromiso o el uso de pseudo pesos permiten normalizar las distancias a los peores valores observados y elegir el punto que mejor se ajuste [13].

La dominancia de Pareto es la métrica esencial para comparar soluciones en los espacios multiobjetivo. En un problema de minimización, se dice que una solución $\mathbf{x}$ domina a otra solución $\mathbf{y}$, denotado como $\mathbf{x} \preceq \mathbf{y}$, si y solo si se cumplen dos condiciones [12]:

1. La solución $\mathbf{x}$ no es peor que $\mathbf{y}$ en ninguno de los objetivos:

$$z_i(\mathbf{x}) \leq z_i(\mathbf{y}), \quad \forall i \in \{1, \dots, K\}$$

2. La solución $\mathbf{x}$ es estrictamente mejor que $\mathbf{y}$ en al menos uno de los objetivos:

$$\exists j \in \{1, \dots, K\} \text{ tal que } z_j(\mathbf{x}) < z_j(\mathbf{y})$$

El frente de Pareto por lo tanto, es la representación en el espacio de las funciones objetivo de todas las soluciones, que pertenecen al conjunto anterior, el conjunto óptimo de Pareto (las que no están dominadas). Este frente sirve para definir el límite teórico del rendimiento de un modelo. Cualquier movimiento a lo largo del frente implica sacrificar un objetivo para mejorar otro. Lo que buscamos por lo tanto es que nuestro algoritmo busque soluciones lo más cercanas al frente real (convergencia) y distribuidas de manera uniforme (diversidad).

Un MOEA (*Multi-Objective Evolutionary Algorithm*) es una variante de los algoritmos evolutivos diseñado para la optimización multiobjetivo. Se trata de una búsqueda poblacional para aproximar el conjunto óptimo de Pareto en una sola ejecución del algoritmo.

Los algoritmos genéticos (GA) son la base para la mayoría de los MOEA. Los GA son métodos de búsqueda inspirados en la teoría evolutiva, operan con una población de soluciones candidatas (cromosomas) que evolucionan a lo largo de varias generaciones. La naturaleza de los GA es perfecta para la optimización multiobjetivo. Los MOEA utilizan las operaciones básicas de los GA, selección, cruce y mutación, para generar nuevas soluciones a partir de las existentes, explorando y explotando el espacio de búsqueda eficientemente [12].

Para que un algoritmo genético pueda manejar múltiples objetivos debe incorporar mecanismos especializados como [14]:

- Fitness Assignment: El uso de un ranking de Pareto para clasificar frentes.
- Preservar la diversidad: Evitar que la población converja a un único punto. Mecanismos como *fitness sharing*, *crowding distance* o celdas de densidad.
- Elitismo: Hay que asegurar que las mejores soluciones no dominadas no se pierdan en generaciones futuras.

Así la optimización multiobjetivo presenta un marco solución para los conjuntos de clasificadores o *ensembles* pasivos en flujos de datos. Dado que estos conjuntos de clasificadores están actualizándose continuamente, podemos guiar estas actualizaciones periódicas por un MOEA que las optimice. Además la diversidad es crítica para los *ensembles* por lo que explorar frentes de Pareto donde existen configuraciones diversas, podría garantizar mayor robustez ante el *concept drift*.

3.8. NSGA-II

El NSGA-II (*Nondominated Sorting Genetic Algorithm II*) es uno de los algoritmos evolutivos multiobjetivo más influyentes y utilizados. Se basa en identificar un frente de soluciones que

representen los mejores intercambios posibles (*trade-offs*) entre los objetivos en conflicto [14].

Utiliza el concepto de dominancia de Pareto como métrica central para comparar y clasificar las soluciones. NSGA-II extiende el principio de la dominancia para manejar restricciones, donde cualquier solución factible se prefiere a una no factible, y entre dos no factibles, elige la que tenga una menor violación total de las restricciones. La ordenación no dominada permite clasificar a la población en diferentes niveles. El algoritmo identifica todas las soluciones no dominadas y les asigna el Rango 1, luego retira las soluciones temporalmente y repite el proceso para encontrar el segundo frente de no dominancia. Así hasta que todos los individuos estén clasificados. Así utiliza el rango como métrica de comparación primaria.

Si dos soluciones tienen rangos distintos se prefiere la solución con menor rango (el Rango 1 se prefiere al Rango 2), al favorecer sistemáticamente a los rangos inferiores empuja a la población hacia el frente de Pareto real.

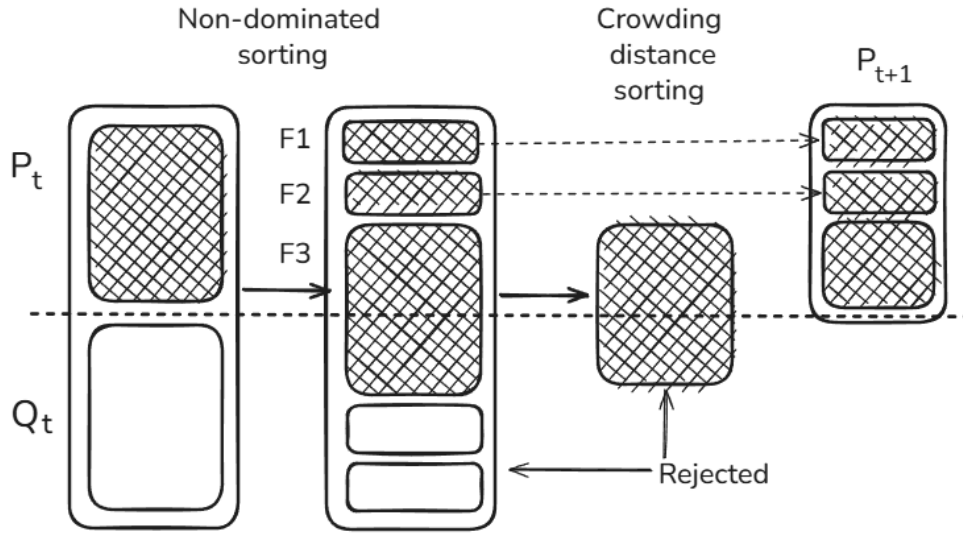


Figura 4: Esquema del NSGA-2.

En cada iteración se combinan la población de padres P_t y la población de descendientes Q_t , generando una población temporal de tamaño $2N$. Para reducir esta población al tamaño original N , el algoritmo selecciona los individuos frente por frente. En primer lugar se incorporan los individuos del frente de rango 1, después los del frente de rango 2 y así sucesivamente [14]. Si al llegar a un frente, no existiera espacio suficiente para añadir todos sus miembros sin superar el tamaño máximo N , se utiliza la distancia de apiñamiento, o *crowding distance*, para seleccionar los individuos más diversos dentro de dicho frente. De esta forma, NSGA-II consigue no solo favorecer las soluciones con mejor rango, sino también las más diversas a lo largo del frente de Pareto.

4. Metodología

4.1. Planteamiento general

La metodología seguida en este trabajo se puede dividir en dos partes. En primer lugar, una evaluación experimental de distintos algoritmos de *ensembles* pasivos diseñados para el aprendizaje en flujos de datos no estacionarios, en concreto los ya expuesto en la sección anterior: SEA, AUE2, WAE y Learn++.NSE. El objetivo de esta primera parte es analizar el comportamiento de varias estrategias de adaptación bajo un mismo conjunto de escenarios, métricas y condiciones. Así poder obtener una comparación de no solo el rendimiento predictivo, sino también la estabilidad, capacidad de recuperación tras los cambios de concepto y el coste computacional de cada uno de los algoritmos planteados.

La segunda fase se basa en los resultados obtenidos en la comparación anterior. Se selecciona bajo los criterios ya expuestos un algoritmo que se utilizará como base para sobre él, diseñar un modulo de optimización multiobjetivo, cuyo objetivo no es sustituir al algoritmo, sino actuar como una capa de ajuste dinámica encargada de modificar su configuración durante el procesamiento del flujo.

En resumen la metodología seguida combina una primera etapa de comparación para seleccionar una base experimental solida y una segunda fase de optimización centrada en si el ajuste dinámico de la configuración del algoritmo obtiene un mejor compromiso entre los objetivos considerados.

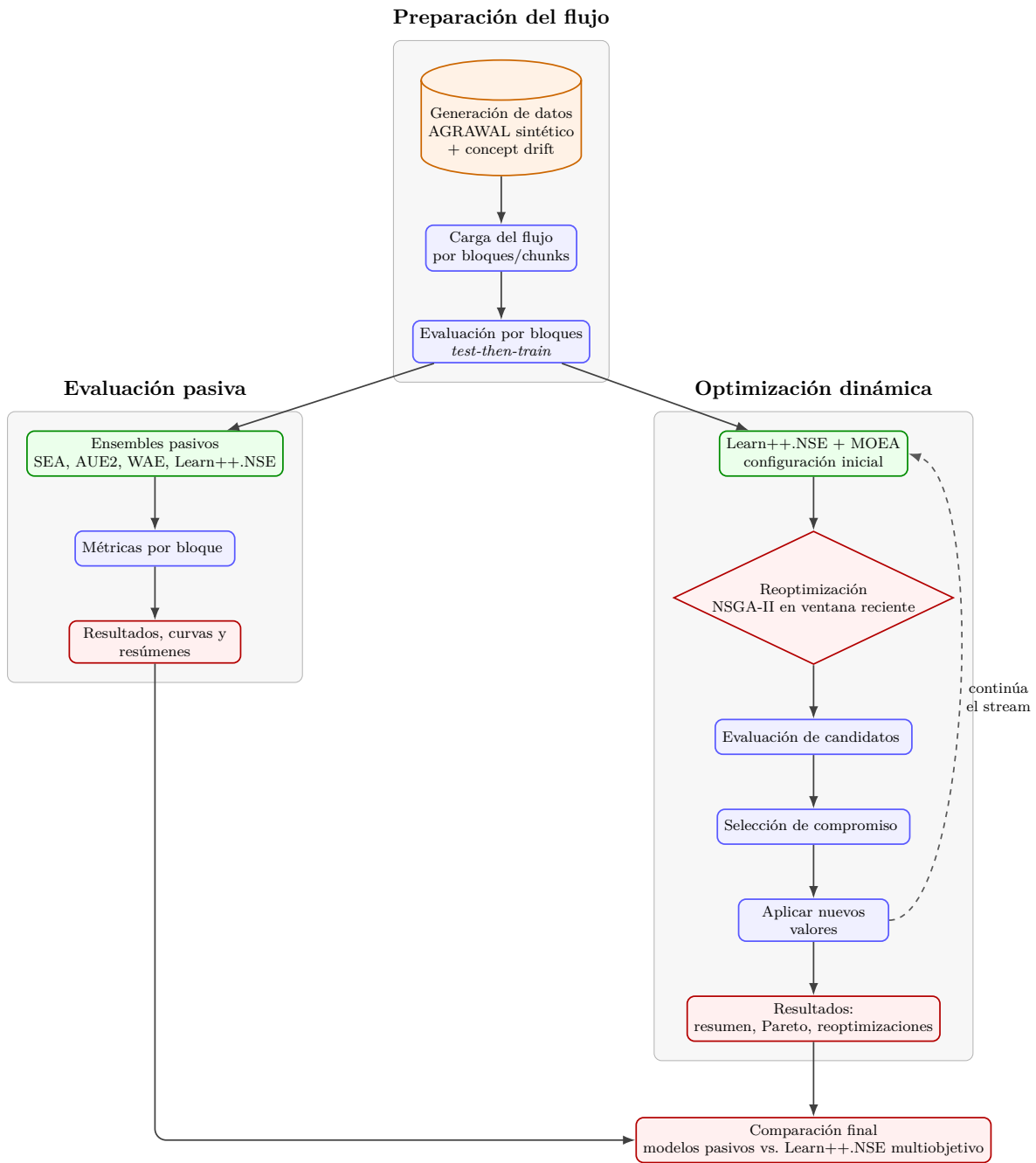


Figura 5: Diagrama de flujo del trabajo.

4.2. Escenarios de evaluación

Para evaluar el comportamiento de los *ensembles* ante cambios en la distribución de los datos, se diseñan varios escenarios de *concept drift*. El objetivo principal de esta evaluación es reproducir en un entorno controlado, las distintas formas en las que puede aparecer la deriva de concepto y así analizar no solo el rendimiento medio de los conjuntos de clasificadores, sino también su capacidad de adaptación cuando el concepto aprendido deja de ser válido, en su totalidad o parcialmente.

Los datasets utilizados en esta evaluación son sintéticos y se generan a partir del generador AGRAWAL, utilizando la librería `river` [15] y la librería `calm-data-generator` [16]. Este generador nos permite simular problemas de clasificación binaria en los que las instancias representan perfiles de clientes, con atributos como salario, edad, nivel educativo, préstamo solicitado, etc. En total hay nueve atributos. En el trabajo, el generador se combina con un generador por bloques para construir los flujos de datos divididos en bloques del mismo tamaño. Los escenarios generados comparten una misma configuración. En total se generan 20.000 instancias, divididas en bloques de 500 muestras, lo que da un total de 40 bloques. Cada dataset se guarda en formato CSV e incluye además de las variables predictoras y el *target*, dos columnas auxiliares: *block*, que indica el bloque temporal al que pertenece la instancia y *concept*, que identifica bajo que concepto se ha generado. Estas dos variables junto con *target* y *timestamp*, se excluyen de los atributos predictivos.

En todos los escenarios se trabaja con dos conceptos distintos. Identificados como concepto A y concepto B. En la configuración utilizada el concepto A corresponde con la función de clasificación 0 que clasifica según la edad y el concepto B la función de clasificación 2 que clasifica según la edad y el nivel educativo. Estos conceptos permiten simular un escenario que podría ser real: a partir de cierto momento se empiezan a mirar el nivel educativo del solicitante para aprobar un préstamo, en vez de solo la edad.

Parámetro	Valor
Generador de datos	AGRAWAL
Herramienta utilizada	<code>calm-data-generator</code>
Número de samples	20.000
Número de bloques	40
Tamaño de bloque	500
Conceptos	<code>classification_fuction</code> 0 y 2
Semilla aleatoria	42
Tipos de deriva	Abrupta, gradual y recurrente

Tabla 1: Configuración de los flujos sintéticos generados.

El primer escenario es el de *dirft* abrupto. En este caso el flujo comienza con una fase estable bajo el concepto A y luego pasa bruscamente al concepto B. Los primeros 15 bloques pertenecen al concepto A y los bloques restantes se generan con el concepto B.

La utilidad del escenario abrupto es que permite medir la capacidad de reacción del modelo ante la pérdida brusca de la validez del conocimiento previo. Resulta relevante ver como de pronunciada es la caída del rendimiento, cuanto tarda el sistema en recuperarse y si los mecanismos de adaptación son capaces de incorporar rápidamente el concepto nuevo. También si el *ensemble* conserva demasiada información antigua o si tiene suficiente plasticidad para ajustarse.

El segundo escenario es el *drift* gradual. A diferencia del caso abrupto aquí el cambio entre

conceptos se produce de forma progresiva durante una ventana de transición. Entre los bloques 12 y 25, las instancias se van mezclando con cierta probabilidad. Para cada instancia se calcula una probabilidad de cambio hacia el concepto B que depende tanto del avance entre bloques como de la propia posición en el bloque. La probabilidad combina un componente asociado al progreso entre bloques y otro componente asociado al progreso dentro del bloque, ponderados respectivamente con un 70 % y un 30 %.

Este escenario es útil porque muchos cambios reales no se producen de manera instantánea. En este contexto, el análisis se centra en comprobar si los *ensembles* pueden adaptarse sin sobrerreaccionar ante pequeñas variaciones iniciales y el equilibrio entre estabilidad y plasticidad: debe conservar el conocimiento útil del concepto A mientras transiciona al B.

El tercer y último escenario es el de *drift* recurrente. En este caso los conceptos se van alternando durante el flujo de datos. La secuencia comienza con 10 bloques del concepto A, 8 bloques del concepto B, vuelve a 6 bloques del concepto A, pasa a 8 bloques del concepto B y termina los últimos bloques con el concepto A. Este escenario simula un comportamiento que podría ser estacional, por ejemplo, ciertos perfiles reaparecen en campañas concretas. . En un entorno recurrente, olvidar completamente el conocimiento pasado puede ser negativo, ya que un concepto descartado podría volver a ser relevante más adelante. Por tanto, este escenario es adecuado para estudiar la memoria del sistema y la reutilización de conocimiento.

Escenario	Descripción del cambio	Objetivo
Abrupto	Cambio repentino entre conceptos	Medir caída y recuperación del modelo
Gradual	Transición progresiva entre conceptos	Evaluar adaptación durante el cambio
Recurrente	Reaparición de conceptos previos	Analizar reutilización de conocimiento

Tabla 2: Escenarios de evaluación utilizados en los experimentos

En definitiva, la evaluación mediante *datasets* sintéticos basados en AGRAWAL proporciona un entorno controlado, reproducible y suficientemente flexible para estudiar el comportamiento de los modelos ante diferentes cambios. Al conocer de antemano cuándo y cómo se produce el *drift*, es posible interpretar con mayor precisión las variaciones en el rendimiento y relacionarlas con la estructura del escenario. Esto permite valorar no solo qué conjunto de clasificadores obtiene mejores resultados globales, sino también qué estrategia resulta más robusta ante cambios abruptos, cuál gestiona mejor las transiciones graduales y cuál aprovecha de forma más eficaz la reaparición de conceptos recurrentes.

4.3. Evaluación prequential y Modelos desarrollados

La evaluación prequential o *test-then-train* es el estándar para la validación de modelos en *data stream mining*. A diferencia de los métodos tradicionales de aprendizaje por lotes este se adapta a la continua evolución de los flujos de información. La idea principal es que cada

nuevo bloque de datos se utiliza primero para evaluar el modelo y después para actualizarlo [5]. Es decir el modelo predice sobre datos que todavía no ha visto durante el entrenamiento, y cuando se calculan las métricas se añade el bloque al aprendizaje.

En este proyecto la lógica *prequential* se aplica a la evaluación de los *ensembles*. Para cada bloque, si no es el primero, los modelos realizan predicciones, se calculan el *accuracy*, Kappa y la diversidad del *ensemble*. Después de la evaluación el modelo se entrena con ese mismo bloque. El primer bloque nunca se evalúa por que el modelo no dispone de conocimiento previo; se utiliza como primera actualización.

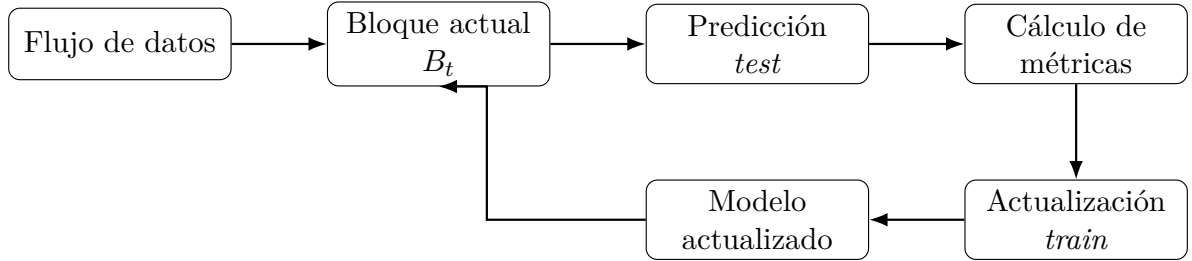


Figura 6: Esquema de evaluación prequential por bloques.

Este mismo enfoque se aplica también a la evaluación del *ensemble* Learn++.NSE optimizado mediante el MOEA, el procedimiento mantiene la misma estructura prequential: primero se evalúa el modelo sobre el bloque actual, después se entrena, y posteriormente, cuando existe suficiente historial, se optimizan los hiperparámetros usando una ventana reciente de bloques.

En este trabajo la evaluación prequential permite construir curvas de rendimiento por bloque y analizar la respuesta temporal de los modelos ante el *drift*. No solo se calcula una métrica global, sino que también información sobre cuándo cae el rendimiento, cuánto tarda en recuperarse y si el modelo mantiene estabilidad frente a cambios sucesivos.

Los modelos que comparamos en este trabajo son:

- SEA
- WAE
- AUE2
- Learn++.NSE

Estos modelos se instancian y evalúan sobre cada dataset y se evalúan con el mismo tamaño máximo para garantizar una comparación homogénea.

En particular se describe ahora el funcionamiento de Learn++.NSE que es el que más adelante se utiliza como baseline para la optimización multiobjetivo. Learn++.NSE mantiene un

conjunto de clasificadores base y un historial de pesos asociados al rendimiento de cada clasificador.

Algorithm 1 Funcionamiento general de Learn++.NSE

```

1: Inicializar el ensemble  $E \leftarrow \emptyset$ 
2: Inicializar el historial de errores  $\beta$  y los pesos de voto
3: for cada bloque  $B_t$  del flujo do
4:   if  $E$  no está vacío then
5:     Obtener las predicciones del ensemble sobre  $B_t$  mediante votación ponderada
6:     Calcular las métricas de evaluación del bloque
7:     Calcular el error del ensemble sobre  $B_t$ 
8:     Construir la distribución  $D_t$  dando mayor peso a las instancias mal clasificadas
9:   else
10:    Inicializar una distribución uniforme sobre las instancias de  $B_t$ 
11:   end if
12:   Entrenar un nuevo clasificador base  $h_t$  con el bloque  $B_t$ 
13:   Añadir  $h_t$  al ensemble  $E$ 
14:   Evaluar los clasificadores de  $E$  sobre  $B_t$  usando  $D_t$ 
15:   Calcular los valores  $\beta$  asociados al error ponderado de cada clasificador
16:   Actualizar el historial de  $\beta$  de cada clasificador
17:   Recalcular los pesos de voto de los clasificadores base
18:   if el tamaño de  $E$  supera el máximo permitido then
19:     Aplicar poda sobre el ensemble
20:     Recalcular los pesos de voto
21:   end if
22: end for

```

4.4. Métricas de evaluación

Para evaluar y comparar los conjuntos de clasificadores ante los distintos escenarios de drift, se utilizan distintas métricas. El objetivo es estudiar la estabilidad, capacidad de recuperación, diversidad interna y el coste del aprendizaje. Las métricas empleadas son: accuracy, accuracy mínima, Kappa, recovery, diversity y coste.

- **Accuracy:** mide la proporción de instancias clasificadas correctamente [1]. En la evaluación *prequential* la calculamos bloque a bloque, después de que el modelo realice las predicciones sobre datos que no han sido utilizados para actualizarlo.

Para un bloque B_t con n_t instancias, se define como:

$$Acc_t = \frac{1}{n_t} \sum_{i=1}^{n_t} \mathbb{I}(\hat{y}_i = y_i)$$

donde y_i es la etiqueta real, $\hat{y}_i$ la predicción del modelo y $\mathbb{I}(\cdot)$ una función indicadora que vale 1 si la predicción es correcta y 0 en caso contrario. A partir de las *accuracies* por bloque se calcula la *accuracy* media:

$$Acc_{mean} = \frac{1}{T} \sum_{t=1}^T Acc_t$$

donde T es el número total de bloques evaluados. Esta métrica permite conocer el rendimiento predictivo general del modelo. En el proyecto, la *accuracy* se obtiene comparando las predicciones con las etiquetas reales de cada bloque.

La *accuracy* mínima representa el peor rendimiento alcanzado durante el flujo:

$$Acc_{min} = \min_{t \in \{1, \dots, T\}} Acc_t$$

- **Kappa de Cohen:** mide el acuerdo entre las predicciones y las etiquetas reales, corrigiendo el acuerdo que podría producirse por azar [17]. Se define como:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

donde p_o es el acuerdo observado y p_e el acuerdo esperado por azar. Kappa resulta útil porque la *accuracy* puede ser engañosa si las clases están desbalanceadas. En el proyecto, esta métrica se actualiza de forma incremental con las predicciones de cada bloque.

- **Recovery:** mide la capacidad de un algoritmo para recuperar su nivel de rendimiento tras una caída provocada por un *concept drift*. Se mide como el número de bloques que transcurren desde el punto de deriva hasta que la precisión del modelo vuelve a estabilizarse [1].

$$Recovery_t = \min\{k \geq 0 : Acc_{t+k} \geq Acc_{rec}\}$$

donde k representa el número de bloques transcurridos desde la aparición de la deriva, Acc_{t+k} es la *accuracy* obtenida en el bloque $t + k$, y Acc_{rec} es el umbral de recuperación definido para considerar que el modelo ha recuperado un rendimiento suficiente.

- **Diversity:** mide el desacuerdo entre los clasificadores base de un ensemble. En este experimento se calcula como desacuerdo medio por pares que es la proporción de instancias en las que un clasificador acierta y el otro falla [17]. Para dos clasificadores i y k , se define como:

$$Dis_{i,k} = \frac{N_{01} + N_{10}}{N_{11} + N_{10} + N_{01} + N_{00}}$$

donde N_{ab} representa el número de instancias según el éxito o fallo de los clasificadores i y k . En concreto, a indica si el clasificador i acierta (1) o falla (0), mientras que b indica si el clasificador k acierta (1) o falla (0). Por tanto, N_{10} recoge los casos en los que el

clasificador i acierta y el clasificador k falla, mientras que N_{01} recoge los casos en los que ocurre lo contrario.

- **Coste:** En el caso de los *ensembles*, el coste se mide como el tiempo total de ejecución necesario para procesar el flujo completo.

$$Cost = T_{exec} = t_{fin} - t_{inicio}$$

donde t_{inicio} representa el instante en el que comienza la evaluación del modelo y t_{fin} el instante en el que finaliza el procesamiento del flujo.

En la versión con optimización multiobjetivo, por un lado, se registra el tiempo dedicado al procesamiento del flujo, es decir, la evaluación y actualización del modelo. Por otro lado, se mide el tiempo consumido por el módulo de optimización. De este modo, el tiempo total puede expresarse como:

$$T_{total} = T_{stream} + T_{opt}$$

donde T_{stream} es el tiempo empleado en procesar el flujo de datos y T_{opt} es el tiempo dedicado a la optimización de hiperparámetros.

4.5. Criterio de selección del algoritmo base

La elección de Learn++.NSE como *baseline* para la optimización multiobjetivo no se hace únicamente atendiendo a la precisión media obtenida durante la evaluación. El criterio de selección considera de forma conjunta el rendimiento predictivo medio y mínimo, la estabilidad del modelo, la capacidad de recuperación, la diversidad del *ensemble* y el coste de procesamiento.

Después de realizar una comparativa que se mostrará más adelante en el trabajo y bajo los criterios de comparación ya expuestos anteriormente, Learn++.NSE es el candidato más adecuado de entre los demás algoritmos: AUE2, WAE y SEA. Además la estructura del algoritmo también es adecuada para el MOEA, ya que Learn++.NSE depende de parámetros que influyen en la ponderación de los clasificadores, la gestión del *ensemble* y la adaptación a nuevos bloques de datos.

En concreto, los parámetros que regulan la función sigmoideal del algoritmo son [4]:

- a : regula la pendiente de la función, determinando con que intensidad se modifican los pesos.
- b : desplaza el punto de transición, que afecta al momento en el que un clasificador empieza a ganar o perder relevancia.

En consecuencia Learn++.NSE se selecciona no solo por el rendimiento experimental sino también por su adecuación metodológica al objetivo principal del trabajo: analizar si una estrategia de optimización multiobjetivo puede mejorar o ajustar el comportamiento de un *ensemble* pasivo en flujos de datos con *concept drift*.

4.6. Módulo de optimización multiobjetivo

El módulo de optimización multiobjetivo tiene el propósito de adaptar dinámicamente la configuración del algoritmo Learn++.NSE en función del comportamiento observado en los bloques recientes. En escenarios con deriva de concepto, por ejemplo una configuración que funciona bien en un periodo estable puede que no ser la más adecuada tras el cambio. Por esto se ha pensado en incorporar este mecanismo.

Como ya se ha dicho en apartados anteriores la idea no es sustituir al algoritmo, si no crear una capa de ajuste, mediante NSGA-II, que mejore su capacidad de adaptación.

Cada individuo del algoritmo evolutivo representa una posible configuración. Es decir, un individuo codifica un conjunto concreto de valores para los hiperparámetros que controlan el comportamiento del *ensemble*.

El espacio de búsqueda del optimizador se define mediante rangos mínimos y máximos para cada uno de estos parámetros. De esta forma, el MOEA no explora configuraciones aleatorias, sino un conjunto acotado de soluciones posibles.

El módulo optimiza tres objetivos de forma simultánea:

1. Busca maximizar la precisión reciente, ya que en entornos con *drift* los bloques más recientes suelen ser más representativos del concepto actual.
2. Busca maximizar la diversidad del ensemble, porque un conjunto de clasificadores diverso puede responder mejor ante cambios en la distribución de los datos.
3. Busca minimizar el coste temporal asociada a la evaluación de la configuración.

En el proyecto, como el optimizador trabaja internamente en forma de minimización, la precisión reciente y la diversidad se introducen con signo negativo, mientras que el tiempo se minimiza directamente.

La optimización se ejecuta durante el procesamiento del flujo, pero no desde el primer bloque. Primero hay que disponer de una ventana completa de bloques recientes sobre la que poder evaluar las configuraciones candidatas. En cada bloque se comprueba si todavía queda flujo por procesar y si existe suficiente historial para formar dicha ventana. Solo cuando se cumplen estas condiciones se activa el proceso de optimización y a partir de entonces se optimiza cada bloque.

Para evaluar una configuración candidata, el sistema toma una ventana reciente de bloques y simula cómo se comportaría Learn++NSE con los hiperparámetros codificados por el individuo. Cada candidato se evalúa creando un modelo Learn++NSE con esa configuración y entrenándolo únicamente sobre la ventana seleccionada. Durante esta evaluación se sigue una lógica *prequential*: si el modelo ya dispone de clasificadores entrenados, primero predice sobre el bloque actual y se calculan la *accuracy* y la diversidad; después, el modelo se actualiza con ese bloque. Una vez evaluadas las configuraciones candidatas, el algoritmo multiobjetivo obtiene un conjunto de soluciones no dominadas o frente de Pareto. Cada solución representa un compromiso distinto entre precisión reciente, diversidad y coste. Como finalmente es necesario aplicar una única configuración al modelo real, se selecciona una solución de compromiso. En el proyecto, esta selección se realiza normalizando los objetivos y aplicando una ponderación que da más importancia a la precisión reciente, seguida de la diversidad y, en menor medida, del coste temporal.

La configuración elegida se aplica al modelo Learn++NSE que está procesando el flujo. Es importante destacar que el modelo no se reinicia ni se sustituyen los clasificadores ya aprendidos. En su lugar, se actualizan los hiperparámetros del *ensemble* para que los siguientes bloques sean procesados con la nueva configuración.

5. Implementación y evaluación experimental

En este capítulo se describe la implementación realizada y se presentan los resultados experimentales obtenidos. Primero se detalla el diseño común seguido por los *ensembles* pasivos implementados. A continuación, se comparan los algoritmos considerados bajo los escenarios de evaluación definidos en la metodología. A partir de esta comparación se selecciona el algoritmo base sobre el que se integra el módulo de optimización multiobjetivo. Finalmente, se presentan los resultados del sistema optimizado y se discuten los resultados obtenidos.

5.1. Diseño e implementación de los ensembles pasivos

En esta parte se va a detallar la implementación de los conjuntos de clasificadores: SEA, AUE2, WAE y Learn++NSE. Recordar que estos métodos se denominan pasivos porque no utilizan un detector de *drift* explícito, se adaptan al flujo incorporando nuevos clasificadores base a medida que llegan nuevos bloques de datos. Todos los *ensembles* pasivos siguen una estructura común basada en dos operaciones principales: `fit_chunk`, para entrenar el modelo con un bloque de datos y `predict`, para obtener las predicciones sobre un bloque. De esta manera podemos evaluar todos los métodos con el mismo procedimiento *prequential*.

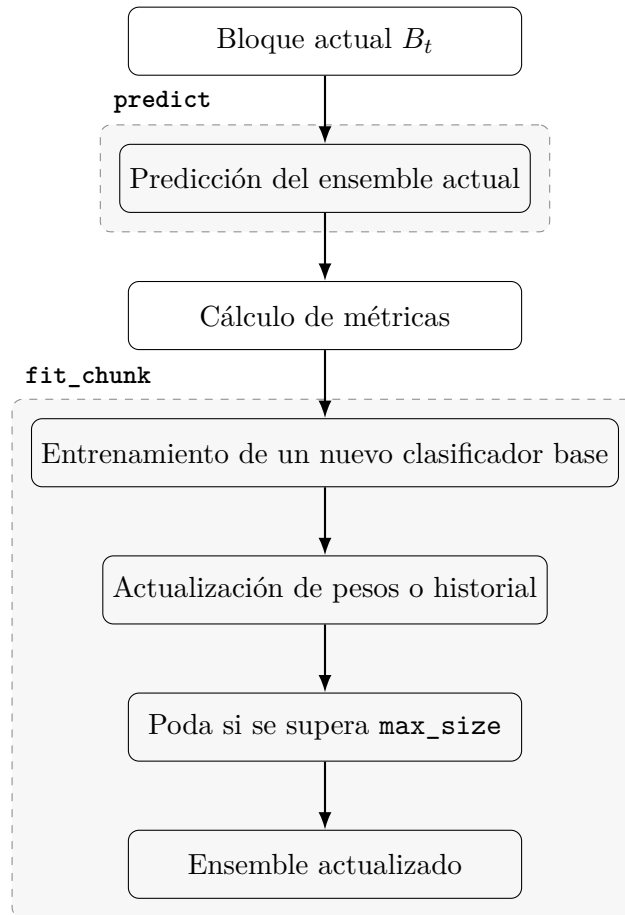


Figura 7: Flujo común de los ensembles pasivos implementados.

Cabe también destacar que se ha utilizado `HoeffdingTreeClassifier` de la librería `river`, como clasificadores base para todos los *ensembles*. El uso de Árboles de Hoeffding o VFDT (*Very Fast Decisions Trees*) es por su arquitectura incremental de una única pasada que permite procesar flujos con memoria limitada y garantiza la convergencia mediante la cota de Hoeffding [18]. Y tienen una naturaleza como aprendices inestables que es necesaria para la diversidad del *ensemble*.

Implementación SEA

Su diseño es el más sencillo de los ensembles implementados. La principal diferencia de SEA es que no utiliza pesos de votación. La predicción se obtiene mediante votación mayoritaria simple. Cuando el ensemble supera el tamaño máximo se aplica una poda basada en rendimiento. Toma una muestra del bloque actual y calcula la accuracy de cada clasificador en esa muestra. El clasificador con peor accuracy es eliminado.

Los hiperparámetros principales de esta implementación son:

- `pruning_sample_frac`: Este hiperparámetro indica la fracción del bloque actual que se

utiliza para decidir que clasificador eliminar cuando el ensemble llega al máximo.

Por defecto el valor es 0.5, esto significa que se utiliza el 50 % de las instancias del bloque, un total de 250, para estimar que clasificador tiene peor rendimiento. Se ha elegido este valor como una decisión de compromiso. Evaluar sobre todo el bloque (1.0) aumentaría el tiempo de ejecución y un porcentaje más bajo puede hacer que la muestra no sea representativa.

- **random_state**: Este parámetro fija la semilla utilizada al seleccionar la muestra del bloque para la poda. Si no se fijara esta semilla, distintas ejecuciones podrían seleccionar subconjuntos diferentes del bloque y, por tanto, eliminar clasificadores distintos.

Implementación AUE2

AUE2 se diferencia en que mantiene una lista de pesos asociados. Antes de implementar el nuevo clasificador, la implementación recalcula los pesos de los modelos existentes. El peso de calcula inversamente proporcional a su error: cuanto mas pequeño es el error, mayor peso se le asigna.

Otro detalle de la implementación es que el nuevo clasificador recibe un peso neutro, definido por la media de los pesos existentes o 1.0 si no hay modelos previos. Esto evita favorecer artificialmente al modelo recién entrenado.

La predicción se realiza por votación ponderada, sumando el peso del clasificador a la clase que predice. Cuando se llega al tamaño máximo se conservan los modelos con mayor peso.

Este algoritmo dispone únicamente de un hiperparámetro, **epsilon**, este parámetro es un vlaor pequeño que se añade al denominador al calcular el peso de cada clasificador. El peso se obtiene como:

$$w_i = \frac{1}{error_i + \epsilon}$$

La función principal que tiene es evitar divisiones por cero, para que el peso no tienda a infinito cuando el clasificador no comete errores sobre el bloque evaluado. Por defecto, el valor utilizado es $\epsilon = 10^{-8}$, que es lo suficientemente pequeño para no alterar los pesos cuando el error es distinto de cero.

Implementación WAE

Esta implementación mantiene una lista de modelos, una lista de iteraciones asociadas a cada clasificador y una lista de pesos de votación. Cuando añade un nuevo clasificador al ensemble calcula una matriz de predicciones base, donde cada fila pertenece a un clasificador y cada columna a una instancia. La matriz se usa para calcular pesos y diversidad.

La estrategia de poda depende de *accuracy* y diversidad. Al llegar al tamaño máximo se simula la eliminación de cada clasificador y se calcula como quedaría el ensemble. La decisión de cual podar se toma maximizando una puntuación combinada.

Los hiperparámetros de este algoritmos son:

- **accuracy_weight**: Controla la importancia de la *accuracy* en la decisión de poda. En la implementación, cuando el ensemble supera el tamaño máximo, se evalúa qué ocurriría si se eliminase cada clasificador. Para cada posible eliminación se calcula una puntuación combinando *accuracy* media y diversidad:

$$score = accuracy_weight \cdot mean_accuracy + diversity_weight \cdot diversity$$

El valor por defecto es:

$$accuracy_weight = 0,7$$

Esto indica que, en la decisión de poda, la *accuracy* tiene más peso que la diversidad.

- **diversity_weight**: Este hiperparámetro controla la importancia de la diversidad del ensemble en la poda. Su valor por defecto es 0.3.

Implementación Learn++.NSE

Esta implementación mantiene una lista de modelos, un historial de valores beta para cada clasificador y una lista de pesos de votación.

A diferencia de SEA, AUE2 y WAE, Learn++.NSE utiliza un mecanismo más elaborado de ponderación basado en el historial de errores de cada clasificador. Para cada bloque, si el *ensemble* ya contiene modelos, se evalúa el rendimiento del conjunto sobre el bloque actual y se construye una distribución de pesos sobre las instancias.

Los pesos de votación se recalculan a partir del historial de betas. Para ello se utiliza una ponderación temporal controlada por los parámetros a y b . Y la poda se realiza por antigüedad.

Los hiperparámetros de este algoritmo son de base:

- a : Controla la pendiente de la función utilizada para ponderar temporalmente el historial de errores de cada clasificador.

Los pesos temporales se calculan mediante:

$$\omega = \frac{1}{1 + e^{a(\text{age}-b)}}$$

donde *age* representa la antigüedad del valor dentro del historial y b controla el desplaza-

miento de la función. Un valor mayor de a hace que la transición entre errores recientes y antiguos sea más pronunciada. Es decir, cuanto mayor es a , más rápido disminuye la importancia asignada a los errores antiguos.

El valor por defecto utilizado es $a = 0,5$ este valor se utiliza como configuración inicial porque introduce una penalización temporal moderada. Valores demasiado bajos harían que la función cambiase muy lentamente, y valores demasiado altos provocarían una transición muy brusca, haciendo que el algoritmo olvidase con demasiada rapidez información que todavía podría ser útil.

- b : El parámetro b controla el desplazamiento de la función de ponderación temporal. En la práctica, ayuda a determinar a partir de qué antigüedad los errores empiezan a perder más peso dentro del historial.

El valor por defecto utilizado es $b = 5$. Representa una ventana de memoria reciente con el tamaño que después utilizará el módulo de optimización. Si el flujo tiene 40 bloques, no se quiere que un error ocurrido hace 30 o 35 bloques tenga el mismo peso que un error reciente.

Algoritmo	Diferencia principal	Parámetros específicos	Valores por defecto
SEA	Votación mayoritaria simple y poda según la peor <i>accuracy</i> reciente.	<code>pruning_sample_frac</code> <code>random_state</code>	0,5 42
AUE2	Votación ponderada según el error reciente de cada clasificador.	<code>epsilon</code>	10^{-8}
WAE	Poda basada en una combinación de <i>accuracy</i> y diversidad.	<code>accuracy_weight</code> <code>diversity_weight</code>	0,7 0,3
Learn++.NSE	Historial de betas, ponderación temporal y poda configurable.	a b	0,5 5

Tabla 3: Tabla resumen de los algoritmos base implementados.

5.2. Comparación de los ensembles pasivos

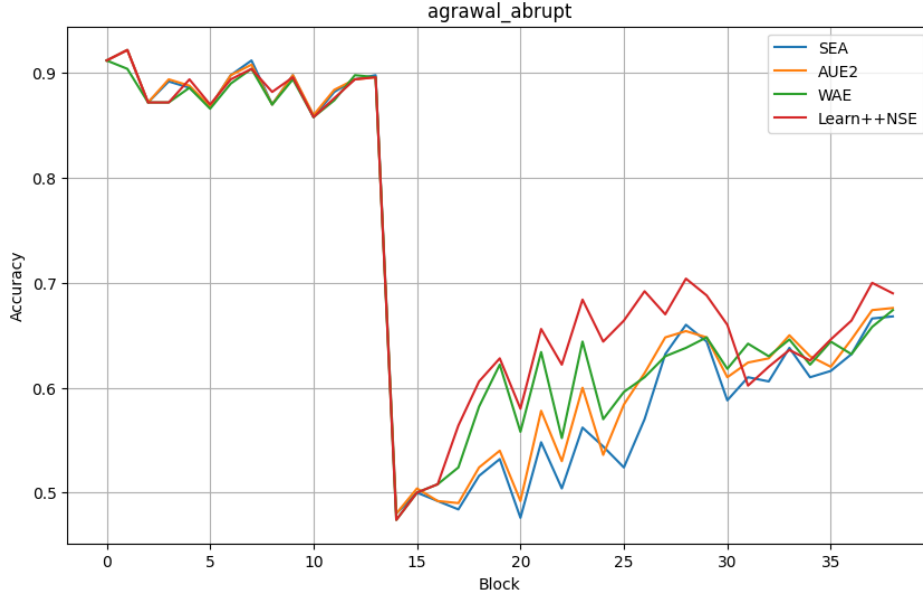


Figura 8: Gráfica de accuracy en el escenario abrupto.

Modelo	Accuracy	Acc. mín.	Kappa	Diversity	Time (s)	Recovery
Learn++NSE	0.7223	0.4740	0.5652	0.2093	11.8608	7
WAE	0.7039	0.4740	0.5365	0.2166	18.6659	10
AUE2	0.6958	0.4800	0.5320	0.2066	17.0324	12
SEA	0.6863	0.4760	0.5225	0.2060	13.5418	13

Tabla 4: Resultados de los ensembles en el escenario abrupto.

En este escenario se observa una caída clara de la *accuracy* tras la introducción del nuevo concepto. Recordar que un *drop* tan grande es normal ya que como ya explicamos en la Sección de Evaluación de escenarios la función de clasificación B introduce más complejidad que la A, por lo que es comprensible que en el momento de producirse el *drift* el modelo cometa tantos errores. Podemos ver que el concepto A es sencillo y los modelos empiezan todos con un porcentaje muy elevado, con valores cercanos a 0.9, aprenden correctamente mientras el concepto es estable. Vemos que en los últimos bloques la recuperación no es tan pronunciada, posiblemente por que los modelos estén llegando al techo de *accuracy* para el concepto más complejo B.

La *accuracy* mínima que se registra es de 0.47, todos los modelos presenta una caída prácticamente igual. Tras esta caída, los modelos empiezan una fase de recuperación en la que se aprecian diferencias más claras. Lear++NSE presenta el mejor comportamiento global, con la mayor *accuracy* media (0.72), el mejor Kappa (0.56) y menor tiempo de recuperación. Además del coste más bajo, bastante por debajo de AUE2 y WAE. WAE obtiene la mejor diversidad media aunque no por una diferencia notable.

Se observa también que alrededor del bloque 29 Learn++.NSE sufre otra caída, esto no se debe a otro cambio de contexto, sino a una oscilación, probablemente los clasificadores con mayor peso que estaban trabajando bien en bloques anteriores, para esos nuevos bloques fallan más y hasta que no se reajustan los pesos la predicción conjunta empeora de forma puntual.

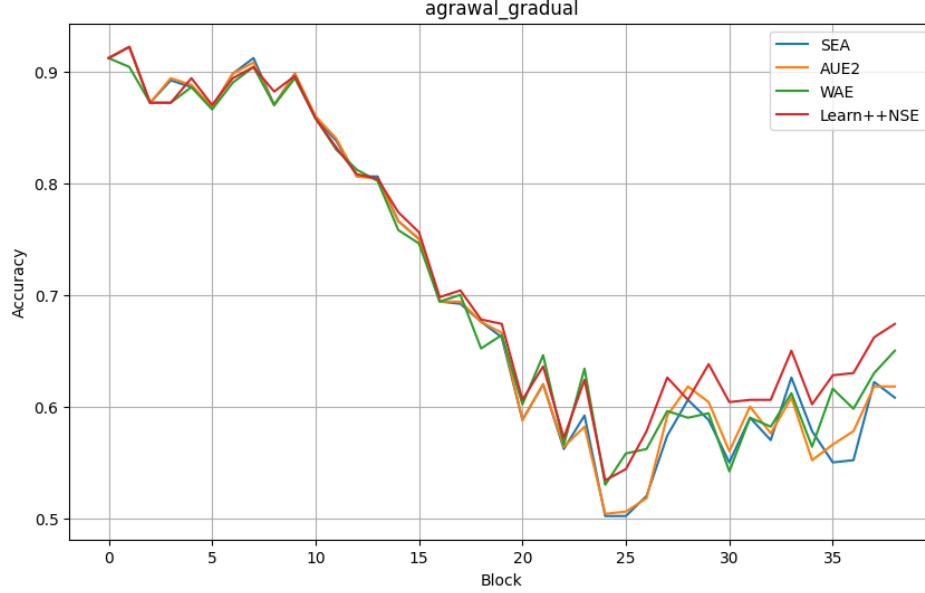


Figura 9: Gráfica de accuracy en el escenario gradual.

Modelo	Accuracy	Acc. mín.	Kappa	Diversity	Time (s)	Recovery
Learn++.NSE	0.7213	0.5340	0.5760	0.1767	10.5338	4
WAE	0.7089	0.5300	0.5535	0.1819	18.0630	11
AUE2	0.7041	0.5040	0.5637	0.1790	16.6058	9
SEA	0.7020	0.5020	0.5618	0.1754	13.0651	8

Tabla 5: Resultados de los ensembles en el escenario gradual.

En este escenario Learn++.NSE vuelve a conseguir el mejor accuracy medio, el mejor Kappa, aunque no por mucha diferencia, la mejor recuperación y de nuevo un tiempo de ejecución considerablemente más pequeño que los demás. Además se observa que tanto Learn++.NSE como WAE consiguen evitar la peor caída de *accuracy* al contrario que en el escenario anterior que todos sufrían igual el cambio de concepto.

WAE obtiene la segunda mejor *accuracy* media, con 0.7089, y destaca por presentar la mayor diversidad del ensemble. Esto es coherente con su diseño, ya que su estrategia de poda tiene en cuenta tanto el rendimiento como la diversidad de los clasificadores. AUE2 y SEA muestran resultados similares en términos de accuracy, aunque SEA destaca por su menor tiempo de ejecución.

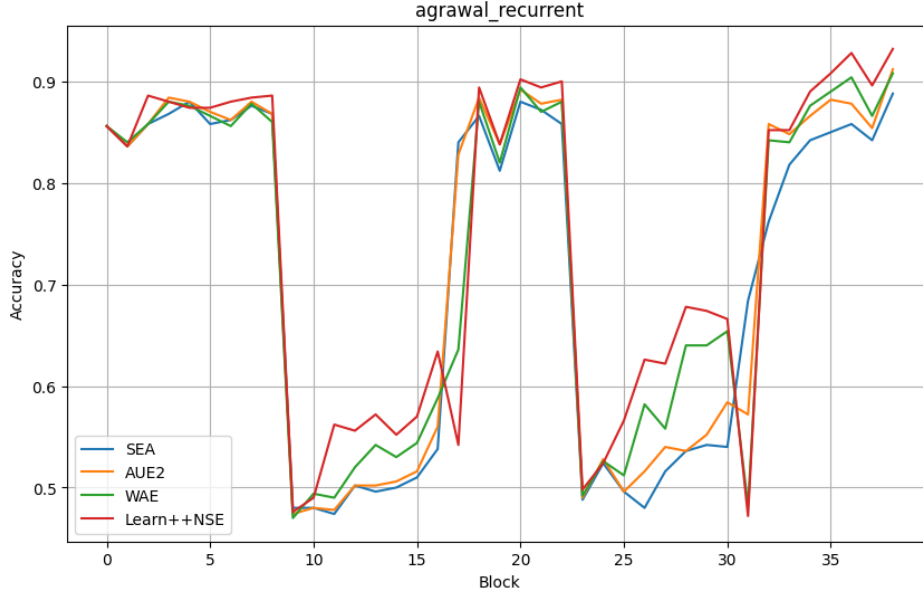


Figura 10: Gráfica de accuracy en el escenario recurrente.

Modelo	Accuracy	Acc. mín.	Kappa	Diversity	Time (s)	Recovery
Learn++NSE	0.7390	0.4720	0.4558	0.2594	11.5068	3.25
WAE	0.7215	0.4700	0.4157	0.2430	18.8597	3.75
AUE2	0.7161	0.4740	0.4084	0.2387	17.0721	5.75
SEA	0.7061	0.4740	0.3951	0.2384	13.6434	6

Tabla 6: Resultados de los ensembles en el escenario recurrente.

Se puede ver que todos los *ensembles* presentan caídas de *accuracy* en los bloques donde se produce un cambio de concepto. Son caídas esperables, tras la caída se observa una fase de recuperación y cuando vuelve el concepto anterior recuperan rápidamente el rendimiento. Para Learn++NSE tras el periodo del concepto B se observa una caída de nuevo de *accuracy* al contrario que en el resto. Esto es debido a que al aparecer el concepto B el algoritmo no es muy conservador y silencia rápidamente a los clasificadores entrenados en el concepto A que están cometiendo errores y hasta que no reajusta para volver a darles la importancia que requiere la situación el *accuracy* cae.

Aunque suceda esto Learn++NSE vuelve a ser por tercera vez el algoritmo que mejor desempeño tiene en todos los aspectos. SEA en este tipo de escenarios al ser el algoritmo más simple sufre más. Además se observa también que tanto Learn++NSE como WAE en la segunda aparición del concepto B reacción antes y obtienen un mejor incremento del *accuracy*.

5.3. Selección del algoritmo base

Escenario	Accuracy	Acc. mín.	Kappa	Diversity	Time (s)	Recovery
Abrupto	Learn++.NSE	AUE2	Learn++.NSE	WAE	Learn++.NSE	Learn++.NSE
Gradual	Learn++.NSE	Learn++.NSE	Learn++.NSE	WAE	Learn++.NSE	Learn++.NSE
Recurrente	Learn++.NSE	AUE2	Learn++.NSE	Learn++.NSE	Learn++.NSE	AUE2

Tabla 7: Algoritmo con mejor resultado por métrica y escenario.

Como muestran los resultados Learn++.NSE obtiene el mejor comportamiento global entre los ensembles pasivos evaluados. Learn++.NSE no solo consigue un buen rendimiento medio, sino que también muestra una mejor capacidad de adaptación ante distintos tipos de drift. En el caso abrupto, se recupera antes tras una caída repentina; en el caso gradual, se adapta mejor durante una transición prolongada; y en el caso recurrente, aprovecha mejor la reaparición de conceptos previamente observados. Por tanto, es el método pasivo más robusto de los evaluados.

Además del rendimiento experimental, Learn++.NSE es un buen candidato para la optimización porque dispone de varios hiperparámetros que influyen directamente en su comportamiento adaptativo. Esto significa que la configuración de Learn++.NSE afecta directamente al equilibrio entre memoria y adaptación. Determinados valores pueden favorecer la conservación de conocimiento antiguo, mientras que otros pueden dar más importancia a la información reciente. Esto encaja perfectamente con el planteamiento del módulo MOEA, que no busca optimizar un único criterio, sino encontrar configuraciones que equilibren varios objetivos y permite un espacio de búsqueda mucho más rico.

Por tanto, la elección de Learn++.NSE se basa en: es el ensemble pasivo que mejores resultados obtiene en los tres escenarios de drift evaluados y su estructura interna basada en historial de errores, ponderación temporal y votación ponderada lo convierte en un algoritmo especialmente sensible a la elección de hiperparámetros. Esta sensibilidad hace que sea un candidato adecuado para aplicar una estrategia de optimización dinámica como NSGA-II.

5.4. Implementación del MOEA

Como ya se ha explicado anteriormente Learn++.NSE se mantiene como clasificador base y el MOEA actúa como un mecanismo externo de búsqueda de configuraciones. Es decir, el optimizador no sustituye al modelo ni reinicia el ensemble, sino que evalúa distintas combinaciones de hiperparámetros y selecciona una configuración de compromiso que se aplica al modelo en ejecución.

Modificación Learn++.NSE

Para permitir la integración con el módulo de optimización, la clase `LearnPPNSE` se amplía con hiperparámetros configurables. La nueva implementación incluye los parámetros `a`, `b`, `grace_period`, `delta` y `weight_power`. Estos parámetros se reciben en el constructor de la clase y quedan almacenados como atributos internos del modelo. A continuación se explican los nuevos parámetros:

- **weight_power**: Este parámetro se introduce para controlar la intensidad de los pesos de votación. Mientras que a y b determinan cómo se pondera temporalmente el historial de betas, `weight_power` actúa a posteriori sobre el peso final del clasificador. De este modo, permite amplificar o suavizar las diferencias entre los pesos de voto del *ensemble*. La idea es:

$$w'_i = w_i^{\text{weight_power}}$$

donde w_i representa el peso original del clasificador i , y w'_i el peso obtenido tras aplicar este parámetro.

- **recency_lambda**: Se utiliza para ponderar la importancia de los bloques recientes durante la evaluación de una configuración candidata. A diferencia de `weight_power`, este parámetro no se aplica directamente dentro de `Learn++.NSE`, sino en el cálculo de las métricas del candidato dentro de la ventana reciente.

Este hiperparámetro permite calcular una media ponderada por recencia. Los valores de los bloques se consideran en orden cronológico y se asigna mayor peso a los bloques más recientes mediante una ponderación exponencial:

$$w_i = e^{-\lambda \cdot \text{age}_i}$$

donde age_i representa la antigüedad del bloque i dentro de la ventana reciente y λ corresponde al valor de `recency_lambda`. Así los bloques más recientes, con menor antigüedad, reciben un peso mayor, mientras que los bloques más antiguos ven reducida progresivamente su influencia.

Si `recency_lambda` = 0 todos los bloques tienen el mismo peso, ya que $e^0 = 1$. En cambio, cuando `recency_lambda` aumenta, los bloques más recientes tienen mayor influencia en la evaluación del candidato.

La inclusión de `grace_period` y `delta` en el cromosoma se debe a que estos parámetros controlan el comportamiento del clasificador base utilizado por `Learn++.NSE`, en este caso el árbol de Hoeffding, `grace_period` y `delta` influyen directamente en la forma en que cada

clasificador base aprende a partir de los datos [15].

- **grace_period**: determina cada cuántas instancias observadas en una hoja se evalúa la posibilidad de realizar una división. Valores bajos pueden hacelerar la adaptación a patrones nuevos, pero harían que sea menos estable. Valores altos hacen que el aprendizaje sea más conservador y menos costoso pero más lento a cambios.
- **delta**: nivel de confianza utilizado en la cota de Hoeffding para decidir si una división es estadísticamente suficientemente fiable. Valores pequeños hacen que el árbol sea más conservador y valores grandes permiten divisiones más rápidas aunque menos robustas.

Por tanto, estos dos hiperparámetros afectan al equilibrio entre plasticidad, estabilidad y coste del clasificador base. Por esto se incorporan al cromosoma del MOEA junto con los parámetros propios de Learn++.NSE. Así el módulo de optimización multiobjetivo ajusta también cómo aprenden los nuevos clasificadores base generados durante el procesamiento del flujo.

Representación del cromosoma

Cada individuo del MOEA es una configuración candidata de Learn++.NSE. El cromosoma tiene seis variables:

$$\mathbf{x} = (a, b, \text{grace_period}, \text{log_delta}, \text{recency_lambda}, \text{weight_power})$$

delta se codifica en escala logarítmica mediante **log_delta**, para poder explorar valores en distintos ordenes magnitud.

El espacio de búsqueda del MOEA se define en la configuración dinámica. Los rangos son:

Parámetro	Rango
<i>a</i>	0,1 – 1,5
<i>b</i>	1,0 – 10,0
grace_period	50 – 150
log_delta	−8,0 – −1,0
recency_lambda	0,5 – 1,5
weight_power	0,5 – 1,5

Tabla 8: Rangos de búsqueda en el cromosoma del MOEA.

Los rangos de búsqueda se han definido alrededor de los valores por defecto de cada hiperparámetro, ampliándolos ligeramente por encima y por debajo. Así el MOEA puede explorar configuraciones alternativas sin aumentar en exceso el espacio de búsqueda ni introducir valores poco razonables.

Para evaluar un individuo, el sistema toma una ventana reciente de bloques y crea un nuevo modelo Learn++.NSE con la configuración codificada por el candidato. Este modelo se entrena

únicamente dentro de esa ventana. Durante esta evaluación se sigue la lógica prequential dentro de la ventana.

Objetivos de optimización

El problema multiobjetivo se define con:

- 6 variables de desión.
- 3 objetivos.

Los tres objetivos son:

1. Maximizar la accuracy reciente.
2. Maximizar la diversidad del ensemble.
3. Minimizar el tiempo de evaluación.

Como `pymoo` formula los problemas de optimización en términos de minimización, los objetivos asociados a la *accuracy* y a la diversidad se introducen con signo negativo.

$$F(\mathbf{x}) = (-accuracy(\mathbf{x}), -diversity(\mathbf{x}), time(\mathbf{x}))$$

donde $accuracy(\mathbf{x})$ representa el rendimiento predictivo obtenido por la configuración candidata, $diversity(\mathbf{x})$ mide la diversidad media del ensemble, y $time(\mathbf{x})$ corresponde al coste temporal de la evaluación.

Solución de Compromiso

Como ya se ha explicado, el resultado del MOEA no es una única solución óptima, sino un conjunto de soluciones no dominadas. Como el frente de Pareto contiene varias soluciones no dominadas, es necesario seleccionar una configuración concreta para aplicarla al modelo real. Para ello, se implementa una función de selección de solución de compromiso.

Dado que el problema se formula en términos de minimización, se trabaja con los objetivos normalizados:

$$\tilde{f}_1(\mathbf{x}), \quad \tilde{f}_2(\mathbf{x}), \quad \tilde{f}_3(\mathbf{x})$$

donde $\tilde{f}_1$ corresponde al objetivo asociado a la *accuracy*, $\tilde{f}_2$ al objetivo asociado a la diversidad, y $\tilde{f}_3$ al tiempo de ejecución. A continuación, se calcula una puntuación ponderada para cada

solución del frente de Pareto:

$$score(\mathbf{x}) = 0,55 \cdot \tilde{f}_1(\mathbf{x}) + 0,30 \cdot \tilde{f}_2(\mathbf{x}) + 0,15 \cdot \tilde{f}_3(\mathbf{x})$$

Los pesos utilizados reflejan la prioridad del sistema: un 55 % para el rendimiento predictivo reciente, un 30 % para la diversidad del ensemble y un 15 % para el tiempo de ejecución. De este modo, el criterio principal de selección es mantener una buena capacidad predictiva, sin ignorar la diversidad del conjunto de clasificadores ni el coste computacional.

5.5. Resultados del sistema optimizado

La siguiente tabla muestra la configuración que se ha utilizado para obtener los resultados:

Parámetro	Valor	Descripción
window_size	5	Número de bloques recientes utilizados para evaluar cada configuración candidata.
pop_size	80	Tamaño de la población utilizada por NSGA-II.
n_gen	50	Número de generaciones ejecutadas en cada proceso de optimización.
max_size	20	Tamaño máximo del ensemble optimizado.
baseline_max_size	20	Tamaño máximo del ensemble usado como configuración base.

Tabla 9: Configuración para la evaluación del módulo MOEA.

Respecto a la población y el número de generaciones se han elegido esos valores siguiendo las indicaciones de [13] y [14]. En el paper original de NSGA-II sugieren que para problemas difíciles se requieren entre 100 y 250 generaciones, sin embargo el coste computacional de ejecutar el experimento con esos parámetros es demasiado alto. En pymoo se habla de que para problemas relativamente simples (como 2 objetivos y 2 restricciones), configuraciones de 40 individuos y 40 generaciones pueden ser suficientes.

Escenario Abrupto

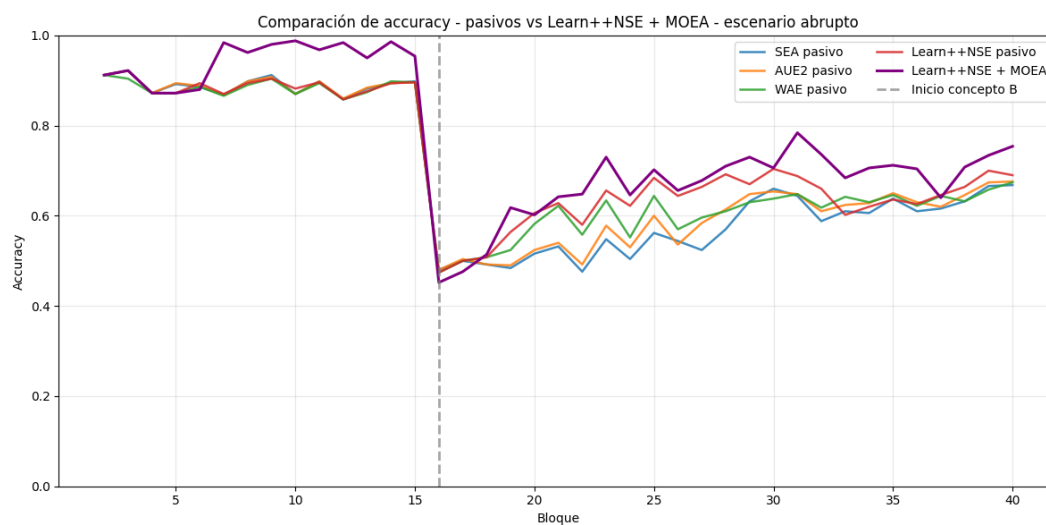


Figura 11: Gráfica comparativa del escenario abrupto.

Modelo	Accuracy	Acc. mín.	Kappa	Diversity	Time (s)	Recovery
MOEA+Learn	0.7663	0.4520	0.6564	0.2559	2754.68	6
Learn++NSE	0.7223	0.4740	0.5652	0.2093	11.8608	7
WAE	0.7039	0.4740	0.5365	0.2166	18.6659	10
AUE2	0.6958	0.4800	0.5320	0.2066	17.0324	12
SEA	0.6863	0.4760	0.5225	0.2060	13.5418	13

Tabla 10: Resultados comparativos del escenario abrupto.

Escenario Gradual

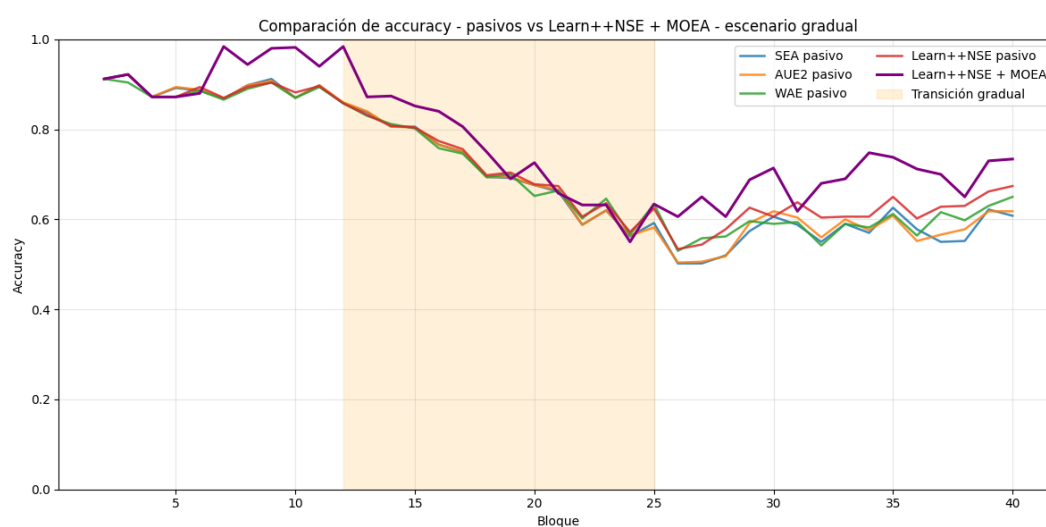


Figura 12: Gráfica comparativa del escenario gradual.

Modelo	Accuracy	Acc. mín.	Kappa	Diversity	Time (s)	Recovery
MOEA+Learn	0.7705	0.5521	0.6732	0.2523	2634.15	1
Learn++.NSE	0.7213	0.5340	0.5760	0.1767	10.5338	4
WAE	0.7089	0.5300	0.5535	0.1819	18.0630	11
AUE2	0.7041	0.5040	0.5637	0.1790	16.6058	9
SEA	0.7020	0.5020	0.5618	0.1754	13.0651	8

Tabla 11: Resultados comparativos del escenario gradual.

Escenario Recurrente

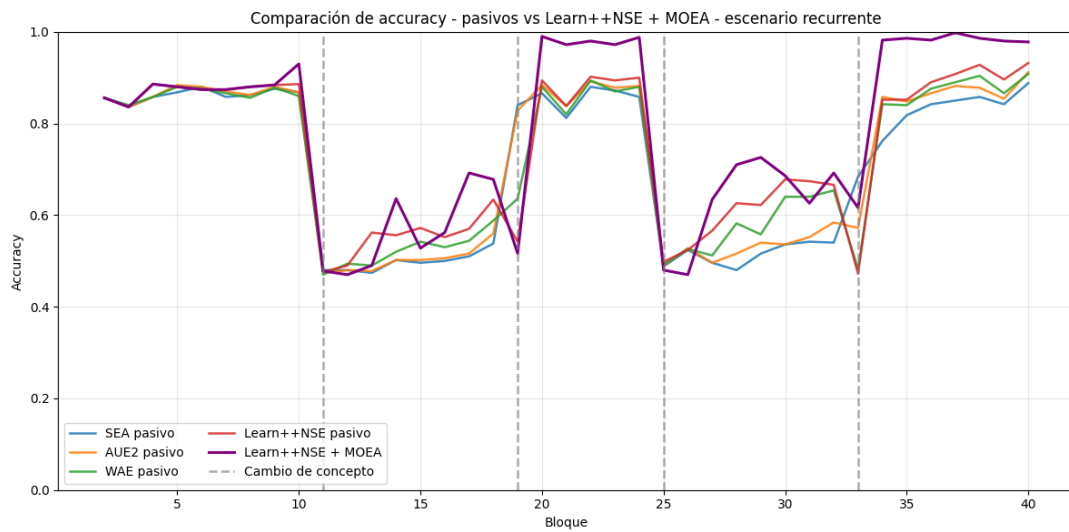


Figura 13: Gráfica comparativa del escenario recurrente.

Modelo	Accuracy	Acc. mín.	Kappa	Diversity	Time (s)	Recovery
MOEA+Learn	0.7705	0.4700	0.5010	0.2799	2812.45	3
Learn++.NSE	0.7390	0.4720	0.4558	0.2594	11.5068	3.25
WAE	0.7215	0.4700	0.4157	0.2430	18.8597	3.75
AUE2	0.7161	0.4740	0.4084	0.2387	17.0721	5.75
SEA	0.7061	0.4740	0.3951	0.2384	13.6434	6

Tabla 12: Resultados comparativos del escenario recurrente.

5.6. Discusión de los resultados

6. Conclusiones

7. Vías futuras

Referencias

- [1] B. Krawczyk, L. L. Minku, J. Gama, J. Stefanowski y M. Woźniak, “Ensemble learning for data stream analysis: A survey”, *Information Fusion*, vol. 37, págs. 132-156, 2017.
- [2] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy y A. Bouchachia, “A survey on concept drift adaptation”, *ACM computing surveys (CSUR)*, vol. 46, n.º 4, págs. 1-37, 2014.
- [3] D. Brzezinski y J. Stefanowski, “Reacting to different types of concept drift: The accuracy updated ensemble algorithm”, *IEEE transactions on neural networks and learning systems*, vol. 25, n.º 1, págs. 81-94, 2013.
- [4] R. Elwell y R. Polikar, “Incremental learning of concept drift in nonstationary environments”, *IEEE transactions on neural networks*, vol. 22, n.º 10, págs. 1517-1531, 2011.
- [5] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama y G. Zhang, “Learning under concept drift: A review”, *IEEE transactions on knowledge and data engineering*, vol. 31, n.º 12, págs. 2346-2363, 2018.
- [6] G. I. Webb, R. Hyde, H. Cao, H. L. Nguyen y F. Petitjean, “Characterizing concept drift”, *Data Mining and Knowledge Discovery*, vol. 30, n.º 4, págs. 964-994, 2016.
- [7] I. Žliobaitė, A. Bifet, J. Read, B. Pfahringer y G. Holmes, “Evaluation methods and decision theory for classification of streaming data with temporal dependence”, *Machine Learning*, vol. 98, n.º 3, págs. 455-482, 2015.
- [8] A. Bifet y R. Gavaldá, “Learning from time-changing data with adaptive windowing”, en *Proceedings of the 2007 SIAM international conference on data mining*, SIAM, 2007, págs. 443-448.
- [9] H. M. Gomes, J. P. Barddal, F. Enembreck y A. Bifet, “A survey on ensemble learning for data stream classification”, *ACM Computing Surveys (CSUR)*, vol. 50, n.º 2, págs. 1-36, 2017.
- [10] W. N. Street y Y. Kim, “A streaming ensemble algorithm (SEA) for large-scale classification”, en *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*, 2001, págs. 377-382.
- [11] M. Woźniak, A. Kasprzak y P. Cal, “Weighted aging classifier ensemble for the incremental drifted data streams”, en *International conference on flexible query answering systems*, Springer, 2013, págs. 579-588.
- [12] A. Konak, D. W. Coit y A. E. Smith, “Multi-objective optimization using genetic algorithms: A tutorial”, *Reliability engineering & system safety*, vol. 91, n.º 9, págs. 992-1007, 2006.
- [13] J. Blank y K. Deb, “Pymoo: Multi-objective optimization in python”, *Ieee access*, vol. 8, págs. 89 497-89 509, 2020.
- [14] K. Deb, A. Pratap, S. Agarwal y T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II”, *IEEE transactions on evolutionary computation*, vol. 6, n.º 2, págs. 182-197, 2002.
- [15] J. Montiel et al., “River: machine learning for streaming data in python”, *Journal of Machine Learning Research*, vol. 22, n.º 110, págs. 1-8, 2021.

-
- [16] Alejandro Belda Fernandez, *calm-data-generator*, <https://github.com/AlejandroBeldaFernandez/Calm-Data-Generator/tree/main>, Repositorio de GitHub. Accedido: 19 de junio de 2026, 2024.
 - [17] L. I. Kuncheva y C. J. Whitaker, “Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy”, *Machine learning*, vol. 51, n.º 2, págs. 181-207, 2003.
 - [18] P. Domingos y G. Hulten, “Mining high-speed data streams”, en *Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining*, 2000, págs. 71-80.